Side Activity 3: Table Creation & Constraints

CSCI3170 Introduction to Database Systems

Learning Objectives

In this side activity, we move from querying data to **building the database structure**:

- Connect to the departmental **Oracle Server** and configure the environment.
- Write `CREATE TABLE` statements with strict constraints (PK, UNIQUE, NOT NULL).
- Implement data logic using `CHECK` constraints.
- Perform specific `INSERT` operations (Full vs. Partial) and handle default values.
- **Optimize the workflow** using SQL*Plus formatting or GUI tools (SQL Workbench).
- Challenge exercise about **Foreign Keys**

Part 1: Connecting to Oracle (The Environment)

Unlike SQLite which lives on your laptop, Oracle is a heavy-duty server. You must log in remotely.

Step 1: SSH into the School Network Open your terminal and run:

```
ssh {your_cse_id}@linux1.cse.cuhk.edu.hk
 # Enter your CSE password when prompted.
```

Step 2: Initialize Oracle Environment Once logged in (you will see a welcome message), run these commands exactly:

```
tcsh
source /opt1/oracle122/setup
```

Step 3: Connect to the Database

```
sqlplus {your_oracle_id}@db18.cse.cuhk.edu.hk
# Enter your Oracle password (check your email if you don't know it).
```

Checkpoint: If you see a prompt like `SQL>`, you are successfully connected.

Part 2: Understanding Constraints via Practice

Instead of reading definitions, let's write code to see how constraints work (and how they fail).

Task 2.1: Basic Constraints (PRIMARY KEY, UNIQUE, DEFAULT)

In this task, we will create a table to manage user accounts. **Important:**

- Oracle uses `VARCHAR2(size)` instead of `TEXT`.

Task: Run the following command to create the table:

```
CREATE TABLE TestUser (
    ID INTEGER PRIMARY KEY,

    -- Oracle requires VARCHAR2(size) for strings.
    Email VARCHAR2(100) UNIQUE,

    -- 'DEFAULT' provides a value if you skip this column during INSERT.
    Status VARCHAR2(20) DEFAULT 'Active'
);
```

Experiment A: Test the DEFAULT value
Insert a user providing *only* the ID and Email.

```
INSERT INTO TestUser (ID, Email) VALUES (1, 'student@cuhk.edu.hk');

SELECT * FROM TestUser;
-- Expected Output: Status should be 'Active'
```

Experiment B: Test the UNIQUE constraint
Try to insert another user with the **same email**.

```
INSERT INTO TestUser (ID, Email) VALUES (2, 'student@cuhk.edu.hk');
```

Observation: You should see `ORA-00001: unique constraint ... violated`.

Task 2.2: Logic Constraints (CHECK)

The `CHECK` constraint ensures data follows a specific rule (e.g., Age cannot be negative).

Run this command to create a table with a rule:

```
CREATE TABLE TestScore (
    ID INTEGER PRIMARY KEY,
    Score INTEGER CHECK (Score >= 0 AND Score <= 100)
);
```

Experiment: Try to break the rule. Attempt to insert a score of 150.

```
INSERT INTO TestScore VALUES (1, 150);
```

You should see an error: "check constraint violated". This confirms your rule is working.

Part Three: Self-Exercises (not graded)

Task 3.1: The Company Table

Write the SQL to create a table named `company` with the following schema:

- **id**: Primary Key (uniquely identifies each company).
- **name**: Cannot be NULL (every company must have a name).
- **address**: Optional (can be NULL).
- **salary**: Must be strictly greater than 0 (Use a `CHECK` constraint).

Task 3.2: The Item Table

Write the SQL to create a table named `item` (referred to as "book" in some slides) with the following schema:

- **book_id**: Primary Key (uniquely identifies each book).
- **title**: Cannot be NULL (every book must have a title).
- **author**: Optional (can be NULL).
- **pub_date**: Optional (can be NULL).
- **isbn**: Cannot be NULL and Must be UNIQUE (no two books with same ISBN).
- **add_date**: Defaults to the current date (`SYSDATE`) if no value is provided.

Task 3.3: Insert Data into Item

Write **three separate INSERT statements** for the `item` table created above:

1. **Full Insert:** Add a new book record by providing values for **all** columns.
2. **Partial Insert:** Add a new book record by providing values for **only some** of the columns, relying on default values or NULL where appropriate.
3. **Minimal Insert (Testing Constraints):** Add a new book record by providing **only** the `book_id` value. *(Note: Since Title and ISBN are NOT NULL, expect this to fail or require adjustment.)*

Part Four: SUPPLEMENTARY TOOLS

This section covers optional tools to improve your database experience. You are encouraged to try these to make your workflow more efficient.

Task 4.1: Beautifying SQL*Plus Output

By default, SQL*Plus assumes a very narrow display width. This causes long rows to "wrap" to the next line, making tables unreadable. Additionally, headers repeat too frequently.

Step 1: Observe the Problem

Run the following query on your newly created employee table (or any system table like `tab`). You will likely see the columns misaligned:

```
SELECT * FROM [Your_Table_Name];;
```

Step 2: Apply Formatting Commands

Run the following commands to configure your session environment. Read the comments to understand what each command does. **These are just examples; don't copy-paste blindly if your table doesn't have a the corresponding column.**

```
-- 1. SET LINESIZE: Controls the width of the line (in characters).
-- Default is 80. Increasing to 200 prevents rows from wrapping.
SET LINESIZE 200;

-- 2. SET PAGESIZE: Controls how many lines print before repeating the header.
-- Default is 14. Setting to 100 means you see headers less often.
SET PAGESIZE 100;
```

```
-- 3. COLUMN Formatting: Adjusts the display width of specific columns.
-- Syntax: COLUMN [ColumnName] FORMAT [FormatModel]
-- 'a20' means "Alphanumeric, 20 characters wide".
COLUMN FULL_NAME FORMAT a20;
COLUMN EMAIL FORMAT a20;

-- You can also format numbers. '9999.99' limits display digits.
COLUMN SALARY FORMAT 99999.99;
```

Step 3: Verify the Result

Run the `SELECT` query from Step 1 again. The output should now be neatly aligned in a single row per record.

Tip: Generalizing this Skill

These settings are temporary for the current session. If you close SQL*Plus, they reset.

- Use `SET LINESIZE` whenever your table rows look "broken".
- Use `COLUMN name FORMAT aXX` whenever a text column is too wide and pushes other data off the screen.

Task 4.2: Configuring SQL Workbench/J (GUI)

If you prefer a Graphical User Interface (GUI) over the command line, you can use **SQL Workbench/J** on your personal laptop. It requires a one-time setup.

1. Prerequisites & Installation

- Ensure you have **Java** installed on your laptop.
- Download **SQL Workbench/J** (Generic package) from the official website and extract the zip file.
- Download the Oracle JDBC Driver (`ojdbc8.jar`) from the Oracle website (this you already have from your first lab worksheet).

2. Setting up the Driver

SQL Workbench needs the Oracle driver to "talk" to the database.

1. Open SQL Workbench/J (run `sqlworkbench.exe` or `sqlworkbench.jar`).
2. Go to **File** → **Manage Drivers**.
3. Select **Oracle** from the list (or create a new entry).
4. Click the **Folder icon** (Library) and select the `ojdbc8.jar` file you downloaded.
5. Click **OK**.

3. Creating the Connection Profile

Now, configure the connection to the school server.

- **Connection Name:** CSE Oracle (or any name you like)

- **Driver:** Select "Oracle" (the one you just configured).
- **URL:** `jdbc:oracle:thin:@db18.cse.cuhk.edu.hk:1521:db18`
- **Username:** Your Oracle username.
- **Password:** Your Oracle password.

Note: You must be connected to the school network (WiFi or VPN) for this connection to succeed.

4. Using the Tool

Once connected, you will see a query window.

- Type your SQL commands in the top pane (e.g., `SELECT * FROM tab;`).
- Press **Ctrl+Enter** (or the "Play" button) to execute.
- Results will appear in the bottom pane in a grid format.

Part 5: Self Challenge Exercises (not graded)

The "Reviews" Table

Create a table named `reviews` to store user ratings for the books in the `item` table.

- **review_id**: Primary Key.
- **book_id**: Number. This should be a **Foreign Key** referencing `item(book_id)`.
- **rating**: Number. Must be an integer between 1 and 5 (Inclusive).
- **comment_text**: Text. Optional.
- **review_date**: Date. Defaults to today.

Hint: You will need to look up the syntax for `REFERENCES` and composite `CHECK` constraints (e.g., `CHECK (rating >= 1 AND rating <= 5)`).

End of side activity 3.